

Intern ID : CITS2440

PROJECT TASK -1 DOCUMENTATION REPORT

**Presented by :
Jahnvi Senagana**

Logic Gates Design Using Verilog

Abstract :

Logic gates are the fundamental building blocks of digital electronics and computer systems. This project focuses on the design and simulation of basic logic gates using Verilog HDL. The implemented gates include AND, OR, NOT, NAND, NOR, XOR, and XNOR gates. The functionality of each gate was verified through simulation using EDA Playground and waveform analysis using EPWave.

Introduction :

Digital systems operate using binary logic where inputs and outputs are represented using logic levels 0 and 1. Logic gates are electronic circuits that perform logical operations on binary inputs.

This project demonstrates the implementation of various basic logic gates using Verilog Hardware Description Language (HDL). The outputs were verified using testbench simulations and waveform generation.

Objective :

The objective of this project is to design and simulate basic logic gates such as AND, OR, NOT, NAND, NOR, XOR, and XNOR using Verilog HDL and verify their outputs through simulation and waveform analysis.

Tools Used ;

- Verilog HDL
- EDA Playground
- Icarus Verilog Simulators
- EPWave Waveform Viewer

Logic Gates Implemented :

1. AND Gate

Produces logic HIGH only when both inputs are HIGH.

2. OR Gate

Produces logic HIGH when at least one input is HIGH.

3. NOT gate

Produces complement of the input signal.

4. NAND Gate

Produces inverted AND output.

5. NOR Gate

Produces inverted OR output.

6. XOR Gate

Produces HIGH output when inputs are different.

Truth Table :

A	B	AND	OR	NAND	NOR	XOR
0	0	0	0	1	1	0
0	1	0	1	1	0	1
1	0	0	1	1	0	1
1	1	1	1	0	0	0

NOT Gate truth table :

A	NOT
0	1
1	0

Verilog Design

The logic gates were implemented using continuous assignment statements in Verilog HDL.

Example:

```
assign y = a & b;
```

The above statement represents AND gate operation.

TestBench

A testbench was written to apply different input combinations to the logic gates and verify the outputs. Simulation delays were used to observe changes in output waveforms.

Simulation

The project was simulated using EDA Playground with the Icarus Verilog simulator. EPWave was used to analyze the generated waveforms.

Simulation successfully verified:

- Correct logical operations
- Proper waveform generation
- Truth table validation

Output Waveform Analysis

The waveform output showed changes in output signals corresponding to different input combinations. The generated outputs matched the expected truth table results for all implemented logic gates.

Learning Outcomes

- Understanding of digital logic fundamentals
- Verilog HDL programming
- Testbench development
- Simulation and waveform analysis
- Digital circuit verification

Conclusion

The Logic Gates Design project was successfully implemented using Verilog HDL. All logic gates produced correct outputs according to their respective truth tables. Simulation and waveform analysis verified the proper functioning of the digital circuits.

TestBench Code, Main Code

The screenshot displays the EDA Playground interface. On the left, a sidebar contains navigation links: "Languages & Libraries", "Testbench + Design", "Tools & Simulators", and "Examples". The main area is split into two panels: "testbench sv" and "design sv".

testbench sv:

```
1 // Common Testbench for Logic Gates
2 module testbench;
3
4 // Inputs
5 reg a;
6 reg b;
7
8 // Outputs
9 wire and_y;
10 wire or_y;
11 wire not_y;
12 wire nand_y;
13 wire nor_y;
14 wire xor_y;
15
16 // Instantiate module
17 logic_gates uut(
18     .a(a),
19     .b(b),
20     .and_y(and_y),
21     .or_y(or_y),
22     .not_y(not_y),
23     .nand_y(nand_y),
24     .nor_y(nor_y),
25     .xor_y(xor_y)
26 );
27
28
```

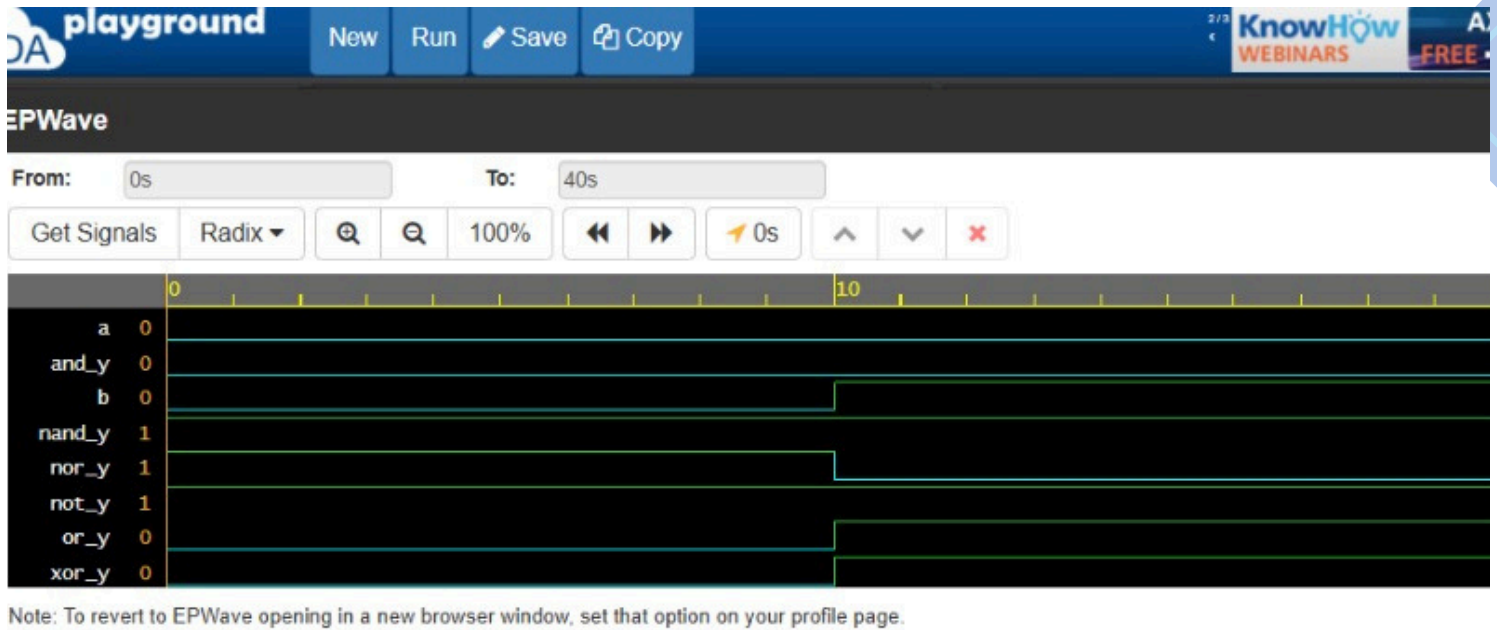
design sv:

```
1 // Common Logic Gates Module
2 module logic_gates(
3
4     input a,
5     input b,
6
7     output and_y,
8     output or_y,
9     output not_y,
10    output nand_y,
11    output nor_y,
12    output xor_y
13 );
14
15 // AND gate
16 assign and_y = a & b;
17
18 // OR gate
19 assign or_y = a | b;
20
21 // NOT gate
22 assign not_y = ~a;
23
24 // NAND gate
25 assign nand_y = ~(a & b);
26
27 // NOR gate
28
```

At the bottom, a "Log" panel shows the simulation output:

```
[2026-06-04 07:08:51 UTC] iverilog '-wall' design.sv testbench.sv && unbuffer vvp a
VCD info: dumpfile dump.vcd opened for output.
A=0 B=0 | AND=0 OR=0 NOT=1 NAND=1 NOR=1 XOR=0
A=0 B=1 | AND=0 OR=1 NOT=1 NAND=1 NOR=0 XOR=1
A=1 B=0 | AND=0 OR=1 NOT=0 NAND=1 NOR=0 XOR=1
A=1 B=1 | AND=1 OR=1 NOT=0 NAND=0 NOR=0 XOR=0
testbench.sv:67: $finish called at 40 (1s)
Finding VCD file...
/dump.vcd
```

Logic Gates Design-EV Waveforms



References :

- Verilog HDL Documentation
- Digital Electronics Fundamentals
- EDA Playground Simulator Documentation